

1. ABSTRACT

A full-stack, microservices-based pharmaceutical assistant that combines Generative AI with structured and unstructured pharmaceutical data. PostgreSQL stores structured entities, MongoDB manages semi-structured documents, and pgvector supports semantic retrieval and RAG. FastAPI and supporting services expose REST APIs, while Kafka, Docker and CI/CD support scalable deployment.

The goal is to provide context-aware, knowledge-grounded answers and reduce unsupported AI responses.

2. PROBLEM STATEMENT & OBJECTIVES

Problem: Keyword search may miss query meaning and context; pharmaceutical information is scattered across sources; structured and unstructured data are difficult to combine.

Objectives:

- Build a reliable pharmaceutical AI assistant.
- Integrate SQL, NoSQL and vector databases.
- Implement semantic search and RAG.
- Reduce hallucinated or unsupported answers.

Scope: Users include pharmacists, clinicians and researchers. Modules include search, RAG pipeline, document ingestion, user management and audit logging.

3. EXISTING VS PROPOSED SYSTEM

Feature	Existing System	Proposed System
Search Capability	Keyword-based search (may miss context)	Semantic + keyword + RAG for accurate results
Data Handling	Mostly structured data	Structured (PostgreSQL) + Semi-structured (MongoDB) + Vector (pgvector)
Information Sources	Limited / single source	Multiple sources with unified knowledge base
Answer Quality	Generic or irrelevant	Context-aware, grounded answers
Limitation / Gap	No semantic understanding, scattered data	Solves using semantic search and RAG with unified KB

4. FEASIBILITY STUDY

- Technical:** Required skills in Python, FastAPI, React, PostgreSQL, MongoDB, Docker and Kafka are available.
- Economic:** Uses open-source stack and free-tier hosting for cost efficiency.
- Operational:** User-friendly interface with easy maintenance and high acceptance.
- Schedule:** Can be completed within the semester timeline.

5. TOOLS & TECHNOLOGIES

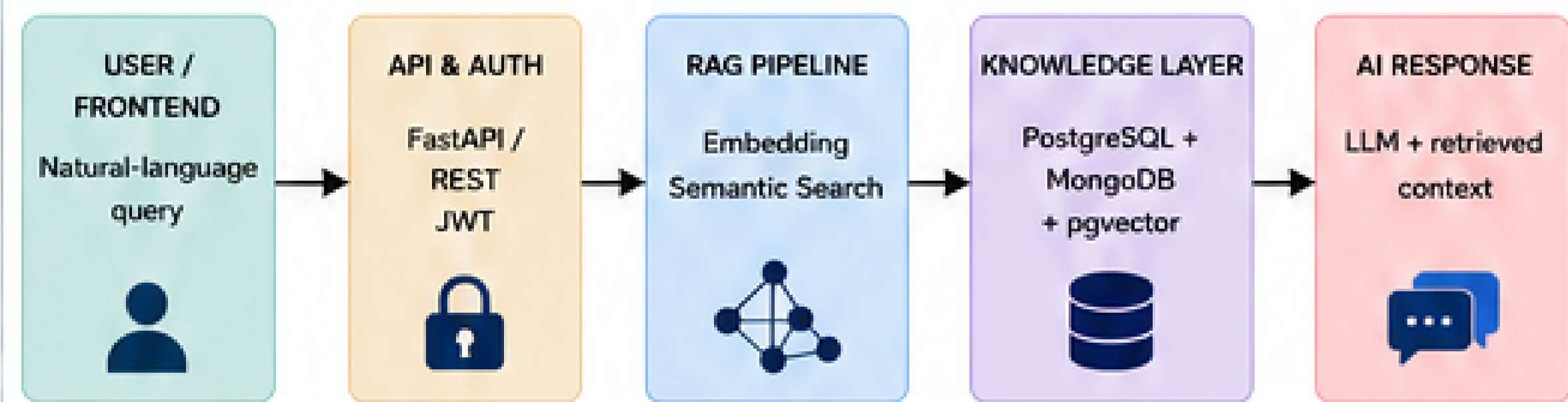
Front-End: React.js, HTML5, CSS3, JavaScript

Back-End: Python FastAPI, Node.js/Express (microservices), REST APIs

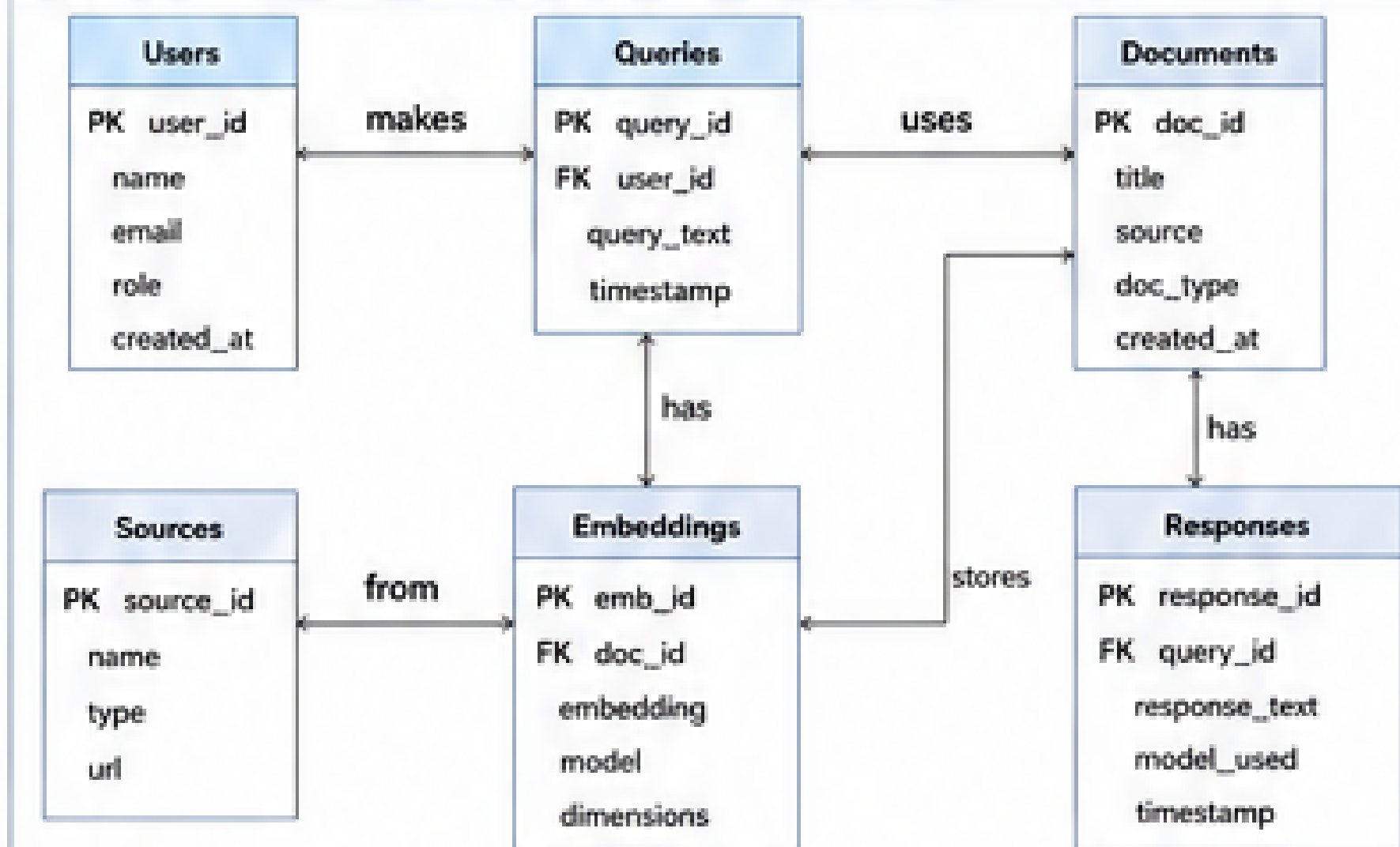
Database: PostgreSQL, MongoDB, pgvector

Tools: Git & GitHub, Docker, Kafka, Postman, VS Code, Swagger, CI/CD (GitHub Actions)

6. SYSTEM ARCHITECTURE

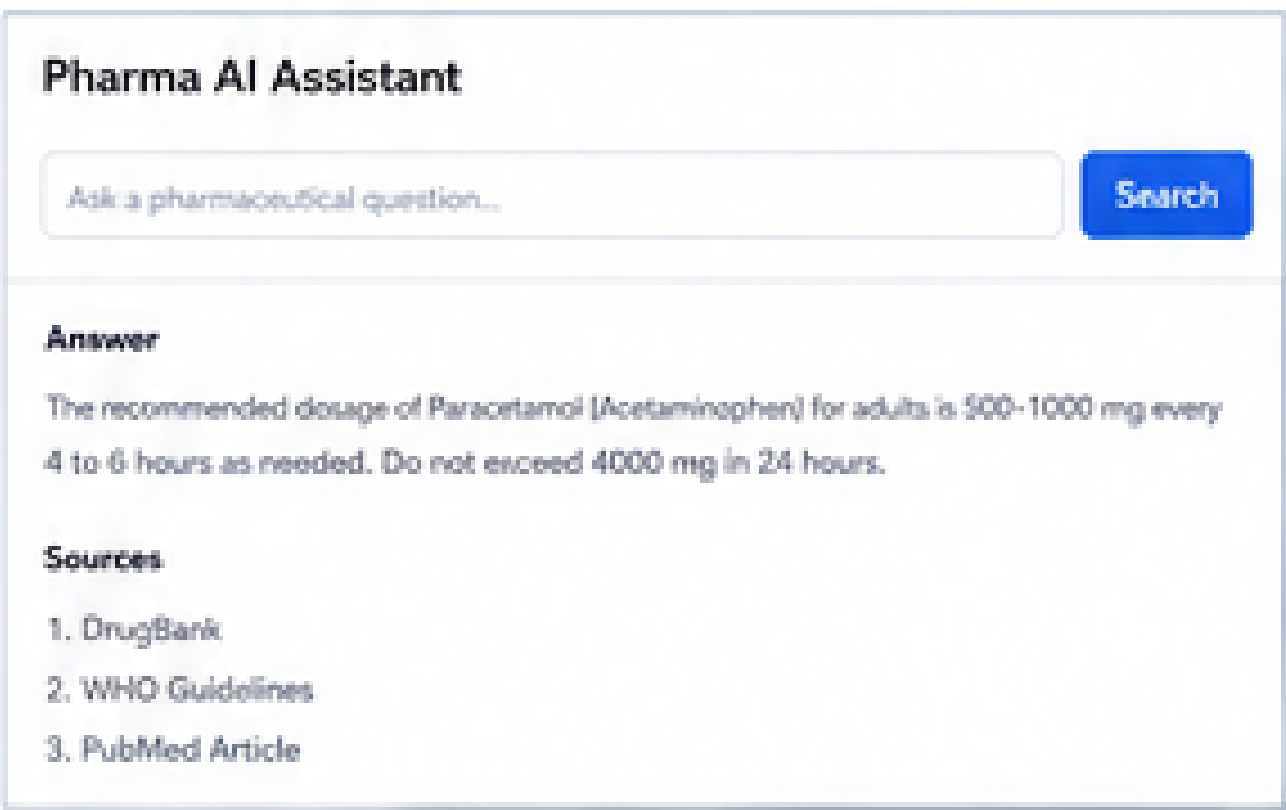


7. ER DIAGRAM & DATABASE DESIGN

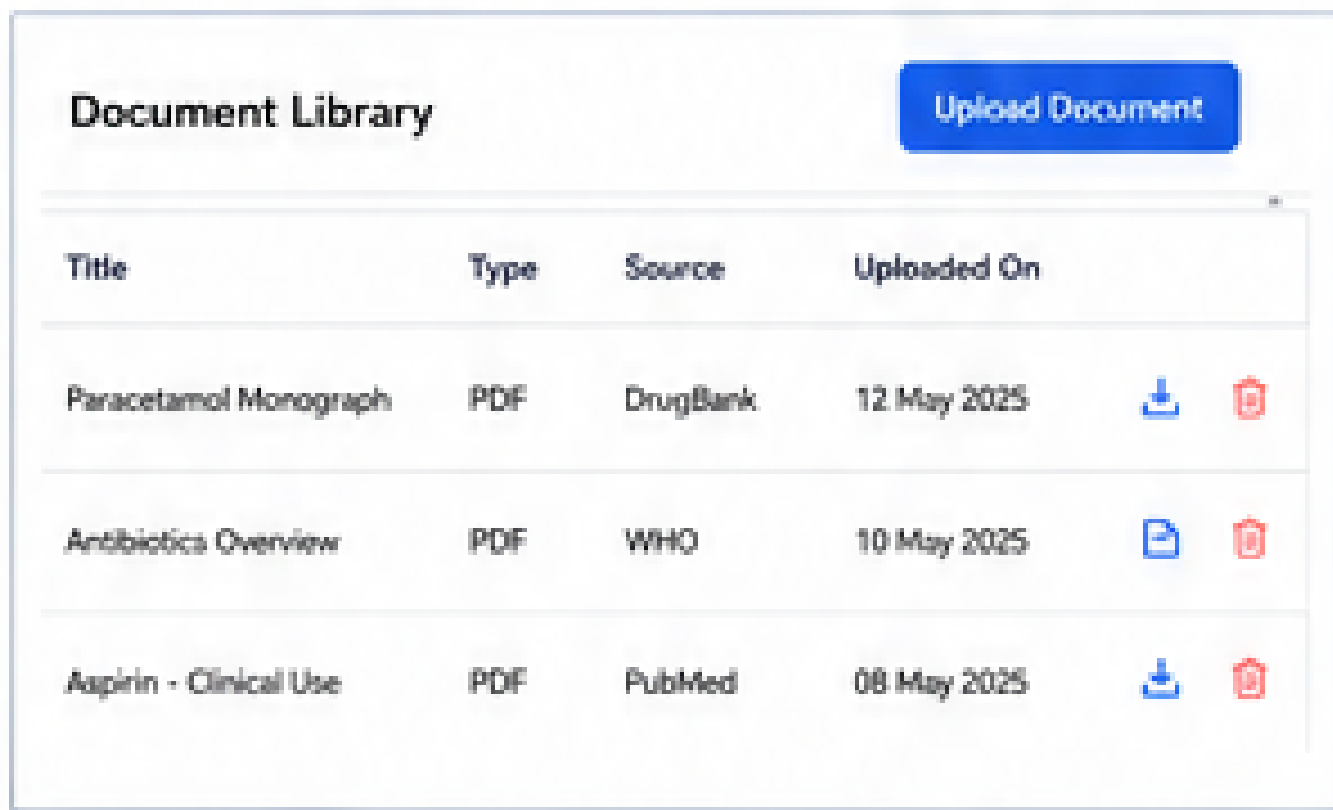


8. IMPLEMENTATION SCREENSHOTS

Screenshot 1 – Search & Results



Screenshot 2 – Document Management



9. RESULTS & TESTING

- Successfully implemented semantic search with RAG pipeline.
- Integrated PostgreSQL, MongoDB and pgvector for unified retrieval.
- Achieved high relevance in answers with reduced hallucinations.
- CRUD operations for documents, sources and users working as expected.
- Response time for average query: ~1.8s
- Test Cases: Functional, Integration, Security and Performance testing completed.

10. CONCLUSION

The project successfully delivers a context-aware pharmaceutical AI assistant using Generative AI and a robust knowledge base. By combining semantic search, RAG and microservices architecture, it overcomes limitations of traditional keyword search systems and delivers accurate, reliable and scalable responses.

All the objectives have been met and the system is **ready for real-world use**.

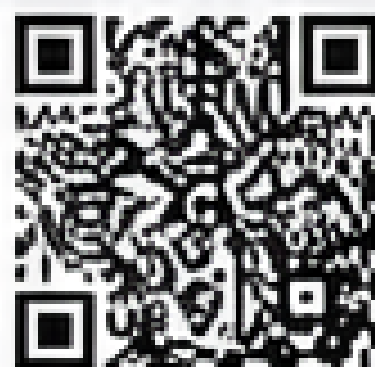
11. FUTURE SCOPE

- Integrate domain-specific medical LLMs for better accuracy.
- Add real-time regulatory data and drug interaction checking.
- Voice-based interaction and multilingual support.
- Stronger encryption, access control and audit logging.
- Cloud deployment and auto-scaling for high availability.
- Continuous monitoring and AI evaluation.
- Support for mobile application.

12. REFERENCES

- Brown, T. et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.
- FastAPI Documentation, <https://fastapi.tiangolo.com/>
- PostgreSQL Documentation, <https://www.postgresql.org/docs/>
- MongoDB Documentation, <https://www.mongodb.com/docs/>
- Lewis, P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

13. QR: GITHUB



Scan to view our
GitHub Repository